

MSC ARTIFICIAL INTELLIGENCE
MASTER THESIS

Scaling On-Policy Distillation Across Extreme Capacity Gaps

by
CARLOS MIGUEL PATIÑO
15485250

June 6, 2026

36 EC
January - June 2026

Supervisors:

Dr E BEECHING
Dr K RASUL
Dr L TUNSTALL
Dr C SNOEK

Examiner:

Dr F PASCOAL DOS SANTOS



UNIVERSITEIT VAN AMSTERDAM



Hugging Face

Contents

1	Introduction	1
2	Related Work	4
2.1	Distillation	4
2.2	Addressing the capacity gap	5
2.3	Context distillation and privileged context	6
3	Methods	8
3.1	Distillation Formalism	8
3.2	Privileged Context	10
3.3	Context Distillation	11
3.4	Top-K KL	12
3.5	Metrics	13
4	Experimental Setup	15
4.1	Data	15
4.2	Models	15
4.3	Inference Engine and Computational Resources	16
4.4	Benchmarks	16
4.5	Implementation	16
5	Experiments	18
5.1	OPD with Privileged Context	18
5.2	Context Distillation vs. Direct System Prompt	20
5.3	Rollout Lengths and Epistemic Words	21
5.4	Using Privileged Context for Teacher-Assisted Distillation	24
5.5	Extending from top-1 to top- K Log Probabilities	24
6	Conclusions	26
A	Appendix	28
A.1	OPD vs Off-Policy Methods	28
A.2	Catastrophic Forgetting	29
A.3	TAID Results	30
A.4	Hyperparameters	30
A.5	Training across Families	30
A.6	Privileged Information Prompts	30

Abstract

Distillation is widely used to make large language model capabilities cheaper to deploy, and has applications in domain-specific assistants, task-focused reasoners, and other systems where a small specialized model is preferable to serving a much larger general model. Recent work has addressed this problem by training students from teacher-generated data, introducing intermediate or adaptive distillation targets, and using on-policy distillation in which a teacher scores rollouts sampled from the current student. Unfortunately, these approaches leave open how distillation behaves when the teacher is hundreds of times larger than the student, a regime where the strongest teachers can also produce supervision that is too complex for the student to learn effectively.

This thesis studies extreme capacity gaps directly and focused on the on-policy distillation setting. On-policy distillation reduces the mismatch between training and inference-time behavior, but it does not by itself solve the problem that small students may receive teacher signals that are too verbose, too sharp, or too specialized to internalize safely. Consequently, we develop and evaluate on-policy distillation variants for mathematical reasoning, including privileged-context teacher scoring, top- k teacher losses, and student-visible context distillation for encouraging concise reasoning. While promising, this regime is non-trivial because on-policy rollouts must be generated inside the training loop, large teachers must provide token-level feedback efficiently, and specialization can cause catastrophic forgetting outside the target domain. Therefore, we further implement scalable training infrastructure for large-teacher on-policy distillation, release it as a TRL trainer, and evaluate target-domain performance, token efficiency, and out-of-domain retention across capacity gaps of up to $400\times$.

Chapter 1

Introduction

Large language models (LLMs) have become increasingly capable general-purpose systems. They can solve a wide range of tasks, adapt to new instructions, and provide useful behavior across domains without requiring task-specific training for every new application. However, this generality comes at a substantial cost because the strongest models are often large, computationally expensive to serve, and slow relative to the requirements of many real-world deployment settings. Even when a large model performs well, using it directly at inference time may be impractical because every query requires substantial memory, compute, and energy.

In many applications, we do not need the full generality of a large language model. Instead, we want a model that is highly capable on a narrower distribution of tasks, such as a domain-specific assistant, a task-focused reasoner, or a model optimized for a particular interaction pattern. For those settings, the ideal system is not necessarily the most general model available, but rather one that preserves the relevant capabilities while being cheap, fast, and reliable to run. This motivates distillation because it enables using a powerful teacher model to transfer useful behavior into a smaller student model [12]. In the best case, distillation allows us to compress the skills of the most capable model into a specialized model that is much more efficient at inference time.

The appeal of distillation is strongest when the student is as small as possible because smaller students are faster to run, easier to deploy, and cheaper to serve. At the same time, the most desirable teachers are usually the largest models available since model capability often increases with scale. This ideal scenario then has a large difference in the number of parameters between the teacher and the student. This difference between the number of parameters is called the capacity gap because it represents a difference in representational and computational power. A large capacity gap is attractive because it promises the greatest efficiency gain: the student is small, while the teacher provides high-quality supervision. However, it also makes the distillation problem substantially harder because the student cannot match the capabilities of a teacher [22, 48].

This difficulty gives rise to what is often called the curse of the capacity gap [48]. When the teacher and student are too different in capacity, the student may fail to benefit from the teacher’s supervision. In extreme cases, a larger, more capable teacher can lead to worse student performance than a smaller, less capable one [22, 19]. This is counterintuitive: one might expect the best teacher to always provide the best training signal. However, if the teacher’s behavior is too complex, too sharp, or too far from what the student can represent, the supervision may become difficult for the student to learn effectively. As a result, the teacher’s additional capability does not translate into better student performance.

This creates a central challenge for distillation. The models we most want to distill from are often the largest and most capable, but those are precisely the ones that may introduce the most severe capacity gap. If we restrict ourselves to smaller teachers, distillation becomes

easier but less desirable because the student inherits behavior from a teacher that is not the best at the task. If we restrict to larger students, we may obtain more capable models at the cost of higher inference costs. To fully realize the promise of distillation, we need methods that enable small students to learn effectively from much larger teachers [32, 34, 15]. Bridging the capacity gap would allow us to combine the strengths of both sides: the capability of large models and the efficiency of small specialized models.

This thesis studies knowledge distillation under extreme teacher–student capacity gaps, with the goal of identifying effective ways to transfer knowledge from large teachers to much smaller students. In particular, we investigate settings in which the teacher can be up to 400 times larger than the student, motivated by the fact that such extreme capacity gaps are becoming increasingly common as the most capable models approach trillion-scale parameter counts [37, 8]. We focus on on-policy distillation (OPD) [1, 9, 20] with privileged context (PC) [25, 50, 31], as this combination is especially promising for addressing large capacity gaps. OPD queries the teacher on states visited by the student, so it reduces the exposure bias that may become more severe as the student’s behavior diverges from that of a much larger teacher. PC provides a complementary benefit because larger teachers are expected to have stronger in-context learning capabilities. As the capacity gap grows, the teacher’s ability to exploit additional context should also increase. Privileged context can therefore leverage the teacher’s growing in-context learning advantage to produce richer and more informative supervision for the smaller student.

To that end, we study OPD under extreme teacher–student capacity gaps and test several ways of making the teacher signal more useful to the student. First, we compare OPD with off-policy distillation and show that on-policy supervision is consistently stronger in our mathematical reasoning setting. However, we also find that OPD alone does not solve the capacity gap: increasing the teacher size from 4B or 30B to 235B does not reliably improve the student and can reduce the fraction of the teacher–student gap that is recovered.

Second, we evaluate privileged context as a way to exploit the teacher’s stronger in-context learning ability. Our results show that this mechanism is fragile. A short PC that informs the teacher about the distillation setup already changes downstream performance, but it does not improve capacity-gap recovery. A more targeted PC for concise reasoning is also less effective than giving the same instruction directly to the student through a system prompt. This suggests that, in on-policy distillation, it is not enough for the teacher to see the right instruction at scoring time; the student must also generate rollouts under the behavior we want it to learn.

Third, we analyze the failures induced by large teachers. We show that truncation is one of the dominant error modes, especially for the 0.6B student trained with larger teachers. We also find that teacher scaling substantially increases rollout length, while the number of epistemic words does not scale in the same way under brevity instructions. This separates two phenomena that are often conflated: longer traces are not simply the result of more explicit epistemic self-talk, but appear to be another symptom of the student struggling to absorb feedback from a much larger teacher.

Fourth, we test whether richer token-level feedback can mitigate the gap by extending OPD from top-1 to top- K teacher log probabilities. Although increasing K improves probability mass coverage, it does not translate into better AIME performance under large capacity gaps. The 0.6B student benefits from larger K only when trained with the nearby 4B teacher; with 30B and 235B teachers, additional log probabilities leave performance unchanged or make it worse.

Finally, studying this regime requires addressing systems challenges that do not appear in ordinary supervised distillation. OPD must generate fresh student rollouts inside the training loop, query large teachers for token-level feedback, and move large log-probability payloads between serving and training workers. To make these experiments possible, we implement

efficient training procedures for large-teacher OPD and release them as a trainer¹ in TRL [38], together with a blog post [24] documenting the engineering lessons.

To summarize, our contributions are:

- Showing that OPD outperforms off-policy SFT across extreme capacity gaps, but that larger teachers do not reliably produce better students or higher gap recovery (Sections 5.1 and A.1).
- Showing that privileged context is not a general solution to the capacity gap: simple teacher-only context can degrade performance, and targeted context distillation is less effective than directly conditioning the student with a system prompt (Section 5.2).
- Identifying truncation and excessive rollout length as central failure modes when small students learn from larger teachers, and showing that rollout-length growth is not explained solely by increased epistemic wording (Sections 5.2 and 5.3).
- Showing that increasing from top-1 to top- K teacher log probabilities improves probability mass coverage but can worsen student performance under large capacity gaps (Section 5.5).
- Developing and releasing an efficient TRL trainer for large-teacher OPD, enabling on-policy rollout generation and remote teacher scoring at the scale required for these experiments.

Together, these contributions aim to better understand the limits of distillation and to expand the range of settings in which small models can successfully learn from much larger, more capable teachers.

¹https://huggingface.co/docs/trl/distillation_trainer

Chapter 2

Related Work

2.1 Distillation

Distillation transfers behavior from a teacher model to a student model by training the student to match the teacher’s predictions or generated outputs. The classical formulation was introduced as a way to compress the knowledge of a large model or ensemble into a smaller model using the teacher’s softened output distribution [12]. In language modeling, the same principle has been adapted to large pretrained models [36], large-scale pretraining systems [39], selective token-weighted objectives [13], and cross-tokenizer teacher–student pairs [6, 21]. For reasoning models, sequence-level and chain-of-thought distillation are also central, with recent work showing that standard supervised fine-tuning pipelines can poorly represent the teacher’s sequence-level distribution and proposing distribution-aligned sequence distillation for long-CoT reasoning [43].

Most classical language-model distillation methods are *off-policy*: the student is trained on a fixed data distribution rather than on rollouts generated by the current student. This fixed data can be human-written, sampled from the teacher, or generated by another system. The supervision can be hard-label supervised fine-tuning (SFT), where the student maximizes the likelihood of target tokens [27], or soft-label distillation, where the student matches teacher log-probabilities or full next-token distributions on the same fixed prefixes. In both cases, the training states are chosen before the student update rather than by the student’s current policy.

Off-policy distillation is attractive because it is simple, stable, and easy to scale. Teacher completions and teacher log-probabilities can be generated once, cached, filtered, and reused across many training runs. This decouples expensive teacher inference from student optimization and makes off-policy distillation a practical baseline for large teachers. Its weakness is that the student is not trained on the prefixes it will actually visit at inference time. For autoregressive models, early student mistakes can move the model into states that are unlikely under the fixed dataset, creating exposure bias [29].

On-policy distillation (OPD) addresses this mismatch by sampling rollouts from the student and asking the teacher to provide feedback at the student’s own prefixes. The teacher no longer only produces complete demonstrations; it gives local supervision, often through next-token log-probabilities, on states induced by the student. Generalized Knowledge Distillation trains language models on self-generated sequences with teacher feedback and improves summarization, translation, arithmetic reasoning, and instruction tuning [1]. MiniLLM argues that reverse KL is better suited to on-policy generative distillation because it discourages the student from assigning high probability to tokens that the teacher considers unlikely [9]. Recent OPD work further shows that useful training signals can be obtained even from sampled-token log-probabilities rather than full vocabulary distributions [20].

This thesis uses OPD as the main training technique because it lets the teacher supervise

the student on the student’s own reasoning trajectories. We nevertheless compare OPD against off-policy distillation methods, because off-policy SFT and teacher-generated traces remain the simplest and most common alternatives. This comparison is especially important in mathematical reasoning, where student errors often create prefixes that are absent from teacher-written solutions. More broadly, on-policy data can mitigate forgetting by preserving the model’s own behavioral distribution during learning [7], and diagnostic work shows that on-policy distillation is most useful when the student rollout is wrong, while the best distillation context depends on both task and student capacity [3].

2.2 Addressing the capacity gap

Distillation is usually motivated by the asymmetry between a larger, more capable teacher and a smaller, more efficient student. This asymmetry is also the source of a central difficulty: a stronger teacher is not always a better teacher for a much smaller student. Early studies [22] show that student performance can degrade when the teacher is too large relative to the student, motivating intermediate teacher-assistant models to bridge the gap [22]. Later work describes this failure mode as the “curse of capacity gap,” where a large teacher’s additional capability does not transfer cleanly to a much smaller student [48]. This problem is central to our setting because the most appealing deployment regime is precisely the hardest one: using the strongest available teacher to train the smallest student that is still cheap and fast enough to serve.

Existing approaches to the capacity gap often introduce additional machinery to make the teacher signal easier for the student to learn. Teacher-assistant methods insert intermediate models between teacher and student [22]. Progressive distillation guides the student along an optimized path from its initial distribution toward the teacher [32]. MiniMoE changes the student architecture by adding minimal expert modules to improve transfer without greatly increasing inference cost [48]. TAID constructs a temporally adaptive intermediate distribution between student and teacher, gradually shifting the target as the student improves [34]. Selective token-weighted distillation masks or down-weights rejected tokens to avoid uniformly applying loss where the teacher signal is noisy or poorly matched to the student [13]. Speculative Knowledge Distillation lets the student propose tokens and the teacher replace low-quality choices, producing training data that is closer to the student’s inference-time distribution [42]. Veto reformulates on-policy targets through a geometric bridge in logit space to suppress harmful gradients when teacher and student are too far apart [15]. In mathematical reasoning, the capacity gap is especially pronounced because small models may fail to benefit from long chain-of-thought traces produced by strong reasoners, and can perform better when trained on shorter or mixed-complexity reasoning traces aligned with their own capacity [19].

These methods show that the capacity gap is not only a question of teacher quality; it is also a question of whether the training setup exposes information that the student can actually use. Our approach is deliberately simpler. We do not add intermediate teachers, change the student architecture, construct adaptive target distributions, selectively mask loss terms, interleave teacher and student sampling, geometrically reformulate the target distribution, or shorten teacher reasoning traces. Instead, we intervene at the prompt level through context supplied to either the teacher or the student, then study whether the resulting behavior can be distilled into the trained student. That way we can keep the standard loss definitions and training loops. This connects the capacity-gap problem to context distillation, privileged context, and self-distillation.

2.3 Context distillation and privileged context

Context distillation refers to training a model so that behavior induced by an explicit context is internalized into the model parameters. The context can be an instruction, explanation, scratchpad, demonstration, optimized prompt, or other conditioning signal that improves behavior when it is present. The goal is to preserve the effect of that context after training, rather than requiring the full context to be supplied at inference time. Early alignment work introduced this idea for long behavioral prompts [4], and subsequent work framed it more generally as distilling useful context tokens such as instructions, explanations, scratchpads, and examples into the model weights [35].

The most direct version is off-policy context distillation. In this setting, the context is associated with the teacher: the teacher receives both the original input and the additional context, produces rollouts under that conditioning, and the student is trained to imitate those rollouts. Knowledge-update distillation follows this pattern by prompting a model with new entity definitions and then training the unconditioned model to match the definition-conditioned behavior [23]. Prompt Baking similarly turns a fixed behavioral prompt into a weight update by making the baked model match the distribution of the prompted model [5]. Generative Prompt Internalization argues that output imitation alone can be an indirect signal for learning the hidden prompt, and therefore also trains the model to generate prompt content and reasons for the behavioral change [33]. These methods differ in how explicitly they expose or reconstruct the hidden context, but they share the same off-policy structure: the training trajectories are generated by a context-conditioned teacher, and the student learns the context only through imitation of those teacher-induced trajectories.

On-policy context distillation introduces the context in two different places. The first route gives the context to the teacher as privileged context. The student generates rollouts from its ordinary input distribution, while the teacher scores the student’s prefixes with access to additional information unavailable to the student. The context is therefore transferred through teacher log-probabilities rather than through teacher-written trajectories. This is the setting studied by On-Policy Context Distillation, which trains the student on its own trajectories while minimizing a reverse-KL objective against a context-conditioned teacher [47]. Privileged-information distillation and related self-distillation methods use the same asymmetry in agentic or reasoning settings: π -Distill and On-Policy Self-Distillation condition the teacher on privileged action information [25], while Self-Distilled Reasoner uses a single model in two roles, with the teacher role conditioned on verified reasoning traces and the student role left unconditioned [50].

A broader self-distillation literature also converts richer feedback into dense token-level supervision by conditioning one role of the model on information unavailable to the other. Self-Distillation Fine-Tuning uses demonstration-conditioned in-context behavior as a teacher for continual learning [31]. Self-Distillation Policy Optimization conditions on textual feedback such as runtime errors or judge evaluations and distills the feedback-informed predictions back into the policy [14]. SD-Zero conditions a reviser on an initial response and binary reward, then distills the reviser into a generator [10]. These methods show that privileged or feedback-conditioned predictions can provide dense supervision, but they usually study self-distillation or teacher-student pairs with relatively matched capacities. They also reveal risks: directly relying on privileged teachers conditioned on reference answers can lead to leakage and unstable training [45, 46], and self-distillation in reasoning can suppress epistemic verbalization and degrade out-of-distribution performance when the context removes useful uncertainty from the teacher behavior [16].

The second on-policy route gives the context directly to the student while it generates rollouts. In this setting, the rollout distribution itself is context-conditioned: the student sees

the instruction or behavioral context during generation, and the teacher then scores prefixes produced by this context-aware student. This differs from privileged-context distillation because the context changes the states visited during training, not only the teacher probabilities assigned to those states. To our knowledge, prior context-distillation work has not studied this student-visible on-policy variant as a way to train the model to internalize an instruction supplied during rollout generation in this capacity gap regime. Both on-policy routes aim to produce a trained student whose behavior incorporates the instruction encoded in the context: the privileged-teacher route transfers the instruction indirectly through teacher log-probabilities, while the student-visible route lets the instruction shape the student’s own training trajectories.

Chapter 3

Methods

3.1 Distillation Formalism

Following the notation of on-policy generalized knowledge distillation [1], let x denote a prompt, let V denote the vocabulary, and let $y = (y_1, y_2, \dots, y_{L_y})$ denote an output sequence. The prefix before token y_n is $y_{<n} = (y_1, \dots, y_{n-1})$. We write the student policy as $\pi_\theta(\cdot \mid x, y_{<n})$ and the teacher policy as $\pi_T(\cdot \mid x, y_{<n})$, both of which define next-token distributions over V .

The simplest distillation objective is SFT, which treats a target sequence as a hard label y_n and trains the student with cross-entropy. If $\mathcal{D}$ is a dataset of prompt-completion pairs, where the completions can be human-written or sampled from the teacher, the loss is

$$\mathcal{L}_{\text{CE}}(\theta) = \mathbb{E}_{(x,y) \sim \mathcal{D}} \left[- \sum_{n=1}^{L_y} \log \pi_\theta(y_n \mid x, y_{<n}) \right]. \quad (3.1)$$

A richer form of distillation uses the full teacher distribution over the vocabulary instead of using the token sampled by the teacher. For a fixed prompt and completion, the token-level forward KL from teacher to student is

$$D_{\text{KL}}(\pi_T \parallel \pi_\theta)(y \mid x) = \sum_{n=1}^{L_y} \sum_{v \in V} \pi_T(v \mid x, y_{<n}) \log \frac{\pi_T(v \mid x, y_{<n})}{\pi_\theta(v \mid x, y_{<n})}. \quad (3.2)$$

The reverse KL swaps the order of the two distributions:

$$D_{\text{KL}}(\pi_\theta \parallel \pi_T)(y \mid x) = \sum_{n=1}^{L_y} \sum_{v \in V} \pi_\theta(v \mid x, y_{<n}) \log \frac{\pi_\theta(v \mid x, y_{<n})}{\pi_T(v \mid x, y_{<n})}. \quad (3.3)$$

The distinction is important because KL divergence is asymmetric. Forward KL (FKL) averages the log-ratio under the teacher distribution, so it penalizes the student for failing to place probability mass on tokens the teacher considers likely. This makes it a mass-covering objective. Reverse KL (RKL) averages under the student distribution, so it penalizes tokens the student samples when the teacher assigns them low probability. This makes it more mode-seeking and can be useful when a lower-capacity student cannot match the full teacher distribution.

We show the distinction graphically in Figure 3.1 using a plot from [9] that shows how the forward and reverse KL fit a Gaussian mixture distribution with a single Gaussian distribution. The reverse KL seeks the major mode of the mixture while the forward KL tries to fit all three.

On top of the distinction between forward and reverse KL, it is important to distinguish where we sample the rollouts y we use for training. Off-policy distillation samples the rollouts from a teacher policy $y \sim \pi_T(\cdot \mid x)$ while OPD samples completions from the student and then

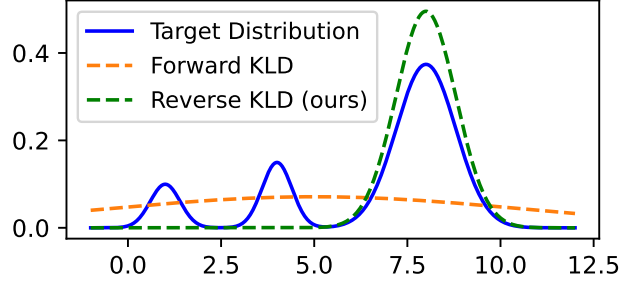


Figure 3.1: Image from [9] showing the difference of using forward and reverse KL to fit a mixture of three Gaussians. The reverse KL tries to fit the major mode while the forward KL tries to approximate the three modes.

compares teacher and student distributions on the prefixes that the student actually visits. We illustrate both settings in Figure 3.2.

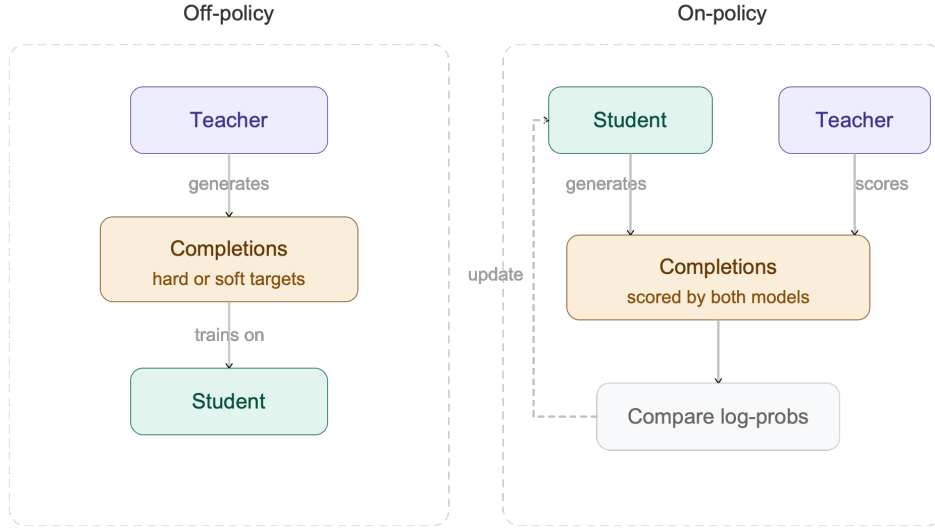


Figure 3.2: Off-policy distillation generates completions from the teacher and trains the student on them. On-policy distillation generates completions from the student and scores them with both models.

The on-policy objective is:

$$\mathcal{L}_{\text{OPD}}(\theta) = \mathbb{E}_{y \sim \pi_{\theta}(\cdot | x)} [D(\pi_T, \pi_{\theta})(y | x)], \quad (3.4)$$

where D can be instantiated as the forward KL in Equation 3.2, the reverse KL in Equation 3.3, or another token-level divergence. We can define off-policy distillation analogously by changing $y \sim \pi_{\theta}(\cdot | x)$ for $y \sim \pi_T(\cdot | x)$.

In practice, computing the full sums over V can be expensive. A common solution is to only use one log-probability instead of the distribution over the full vocabulary. The sampled-token reverse-KL formulation used in on-policy distillation keeps only the realized token at each position of a student rollout $\hat{y} \sim \pi_{\theta}(\cdot | x)$. The resulting top-1 sampled-token estimate is

$$\hat{D}_{\text{KL}}^{\text{top-1}}(\pi_{\theta} || \pi_T)(\hat{y} | x) = \sum_{n=1}^{L_{\hat{y}}} [\log \pi_{\theta}(\hat{y}_n | x, \hat{y}_{<n}) - \log \pi_T(\hat{y}_n | x, \hat{y}_{<n})]. \quad (3.5)$$

This is the sampled-token analogue of Equation 3.3. Rather than matching the full next-token distributions, it scores the token actually produced by the student under both policies.

We can write an analogous formulation for the Equation 3.2, where we only use the realized token at each position of a teacher rollout $\hat{y} \sim \pi_T(\cdot | x)$. Additionally, we use the forward instead of the reverse KL to avoid optimizing a biased surrogate or using importance sampling to correct the off-policy nature of the data.

$$\hat{D}_{\text{KL}}^{\text{top-1}}(\pi_T \| \pi_\theta)(\hat{y} | x) = \sum_{n=1}^{L_{\hat{y}}} [\log \pi_T(\hat{y}_n | x, \hat{y}_{<n}) - \log \pi_\theta(\hat{y}_n | x, \hat{y}_{<n})]. \quad (3.6)$$

The formulation in Equation 3.6 is equivalent to the soft version of SFT, because we are including the log probability of the token sampled by the teacher instead of excluding it and using it as a hard target as in Equation 3.1.

Regardless of the loss function we choose, we can specify what the teacher and student see during training to change the behavior of the trained student.

3.2 Privileged Context

Privileged context is additional information that is available to the teacher during training but will not be available to the student at inference time. Following the privileged information distillation setting [31, 14, 25, 50], the key asymmetry is that the teacher is conditioned on this extra information, while the student is not. Using the notation of Section 3.1, let c denote the privileged context associated with a prompt x . This context can take different forms, such as a reference solution, a successful action trace, tool calls, or a hint extracted from a stronger model. The student policy remains

$$\pi_\theta(\cdot | x, y_{<n}), \quad (3.7)$$

while the privileged teacher policy is evaluated as

$$\pi_T(\cdot | x, c, y_{<n}). \quad (3.8)$$

The goal is therefore not to teach the student to depend on c , but to use c to construct a more informative teacher signal during training. At inference time, the student still acts according to $\pi_\theta(\cdot | x, y_{<n})$.

We can express privileged-context distillation by replacing the teacher distribution in the KL objective with the context-conditioned teacher. For an on-policy rollout $\hat{y} \sim \pi_\theta(\cdot | x)$, the reverse-KL form becomes

$$D_{\text{KL}}(\pi_\theta \| \pi_T^c)(\hat{y} | x, c) = \sum_{n=1}^{L_{\hat{y}}} \sum_{v \in V} \pi_\theta(v | x, \hat{y}_{<n}) \log \frac{\pi_\theta(v | x, \hat{y}_{<n})}{\pi_T(v | x, c, \hat{y}_{<n})}, \quad (3.9)$$

where π_T^c denotes the teacher distribution conditioned on the privileged context c . The corresponding training objective is

$$\mathcal{L}_{\text{PCD}}(\theta) = \mathbb{E}_{(x,c) \sim \mathcal{D}_{\text{PI}}} [\mathbb{E}_{\hat{y} \sim \pi_\theta(\cdot | x)} [D_{\text{KL}}(\pi_\theta \| \pi_T^c)(\hat{y} | x, c)]] , \quad (3.10)$$

where $\mathcal{D}_{\text{PI}}$ is a dataset of prompts paired with privileged context. This objective keeps the rollout distribution on-policy with respect to the student, but computes the teacher probabilities using the stronger context (x, c) . In the self-distillation case, the teacher and student can share parameters and differ only in their input context. In our more general notation and in this

thesis, π_T is a separate teacher model. The defining property of privileged context is that only the teacher receives c .

As in Equation 3.5, a top-1 sampled-token approximation keeps only the token actually produced by the student:

$$\widehat{D}_{\text{KL}}^{\text{top-1}}(\pi_\theta \| \pi_T^c)(\hat{y} \mid x, c) = \sum_{n=1}^{L_{\hat{y}}} [\log \pi_\theta(\hat{y}_n \mid x, \hat{y}_{<n}) - \log \pi_T(\hat{y}_n \mid x, c, \hat{y}_{<n})]. \quad (3.11)$$

This formulation makes the role of privileged context explicit. The sampled token and student probability are computed without access to c , while the teacher probability is computed with access to c . The loss therefore transfers the behavior induced by the privileged context into a student policy that must operate without it.

3.3 Context Distillation

The privileged-context formulation above gives one use of the context c : the teacher sees c , the student does not, and the loss transfers the behavior induced by c through teacher log-probabilities. More generally, context distillation refers to training procedures in which c contains an instruction or other conditioning signal whose effect we want the trained student to incorporate. The possible mechanisms depend on whether the rollout distribution is off-policy or on-policy.

In off-policy distillation, the rollouts are generated before training by the teacher. Since the student does not generate the training states, the only way for the context to affect training is through the teacher prompt used to produce the completions. The context-conditioned rollout is

$$\hat{y}^c \sim \pi_T(\cdot \mid x, c), \quad (3.12)$$

and the student is trained to imitate this fixed completion. This is the same mechanism as SFT on teacher-generated traces in Equation 3.1, except that the traces come from the context-conditioned teacher. If teacher log-probabilities are also stored, the sampled-token forward-KL form from Equation 3.6 can be used instead. In either case, off-policy context distillation transfers c only through the teacher rollout and the student learns to mimic outputs that were generated under the instruction in c .

On-policy distillation has two distinct ways to introduce the same context. The first is the privileged-context setting from Section 3.2. The student generates an ordinary on-policy rollout

$$\hat{y} \sim \pi_\theta(\cdot \mid x), \quad (3.13)$$

but the teacher scores each prefix with access to c . Using the top-1 reverse-KL approximation, this corresponds to the privileged-context objective in Equation 3.11.

Here c is distilled through the teacher probabilities. The student never observes the context directly, but it is updated toward token choices that the teacher prefers when the teacher is conditioned on c .

The second on-policy mechanism is to give the context directly to the student during rollout generation. In this case the rollout distribution itself becomes context-conditioned:

$$\hat{y}^c \sim \pi_\theta(\cdot \mid x, c). \quad (3.14)$$

The same on-policy divergence from Equation 3.4 can then be applied to the tokens produced by this context-aware student, with the teacher evaluated on the same context-conditioned prefixes. In this variant, c changes the states visited during training because the student

generates its own rollouts while aware of the context. This differs from privileged-context distillation, where the rollout distribution is still $\pi_\theta(\cdot | x)$ and only the teacher scores with c . Both on-policy variants can produce a trained student whose behavior reflects the instruction in c without incorporating the instruction explicitly during inference.

3.4 Top-K KL

The KL objectives in Equations 3.2 and 3.3 require summing over the full vocabulary at every token position. This is expensive for large language models, because the vocabulary can contain hundreds of thousands of tokens. The other end of the spectrum is only using the one log-probability as shown in Equations 3.5 and 3.6. A practical approximation that balances these approaches is to keep the k tokens with the largest log-probabilities under one of the two policies. However, we do not renormalize the retained log-probabilities over only those k tokens. Renormalization would discard how much probability mass the model assigns to the rest of the vocabulary. Instead, we group all tokens outside the selected top- k set into a single tail bucket.

For compactness, write $h_n = (x, y_{<n})$. For each prefix, define the teacher and student top- k sets as

$$S_{T,n}^k = \text{TopK}_k(\pi_T(\cdot | h_n)), \quad S_{\theta,n}^k = \text{TopK}_k(\pi_\theta(\cdot | h_n)). \quad (3.15)$$

Given a distribution q and a selected token set S , let τ_S denote the tail bucket containing all tokens in $V \setminus S$. This induces a coarsened distribution $q^{[S]}$ over $S \cup \{\tau_S\}$:

$$q^{[S]}(b | h_n) = \begin{cases} q(b | h_n), & b \in S, \\ 1 - \sum_{u \in S} q(u | h_n), & b = \tau_S. \end{cases} \quad (3.16)$$

This construction preserves a coarse notion of the tail probability that is ignored by the top- k token list. For example, with $k = 10$, the first ten buckets are the ten selected tokens and the eleventh bucket is the total probability mass assigned to every other token. This has three benefits over renormalizing the selected tokens. First, it keeps the top- k probabilities on their original scale, so a model that assigns 90% probability mass to the selected tokens is not treated the same as a model that assigns only 40% mass to them. Second, it allows the loss to penalize disagreements about how much probability should remain in the long tail, even though the individual tail tokens are not represented. Third, it avoids artificially inflating the selected tokens when the model is uncertain and spreads probability mass across many low-probability alternatives.

The standard top- k forward KL uses the teacher top- k set, because the expectation in the forward KL is taken under the teacher distribution. Both the teacher and student are evaluated on the same teacher-induced buckets $S_{T,n}^k \cup \{\tau_{S_{T,n}^k}\}$:

$$\hat{D}_{\text{KL}}^k(\pi_T || \pi_\theta)(y | x) = \sum_{n=1}^{L_y} \sum_{b \in S_{T,n}^k \cup \{\tau_{S_{T,n}^k}\}} \pi_T^{[S_{T,n}^k]}(b | h_n) \log \frac{\pi_T^{[S_{T,n}^k]}(b | h_n)}{\pi_\theta^{[S_{T,n}^k]}(b | h_n)}. \quad (3.17)$$

Analogously, the standard top- k reverse KL uses the student top- k set, because the expectation in the reverse KL is taken under the student distribution. The teacher is evaluated on the same student-induced buckets $S_{\theta,n}^k \cup \{\tau_{S_{\theta,n}^k}\}$:

$$\hat{D}_{\text{KL}}^k(\pi_\theta || \pi_T)(y | x) = \sum_{n=1}^{L_y} \sum_{b \in S_{\theta,n}^k \cup \{\tau_{S_{\theta,n}^k}\}} \pi_\theta^{[S_{\theta,n}^k]}(b | h_n) \log \frac{\pi_\theta^{[S_{\theta,n}^k]}(b | h_n)}{\pi_T^{[S_{\theta,n}^k]}(b | h_n)}. \quad (3.18)$$

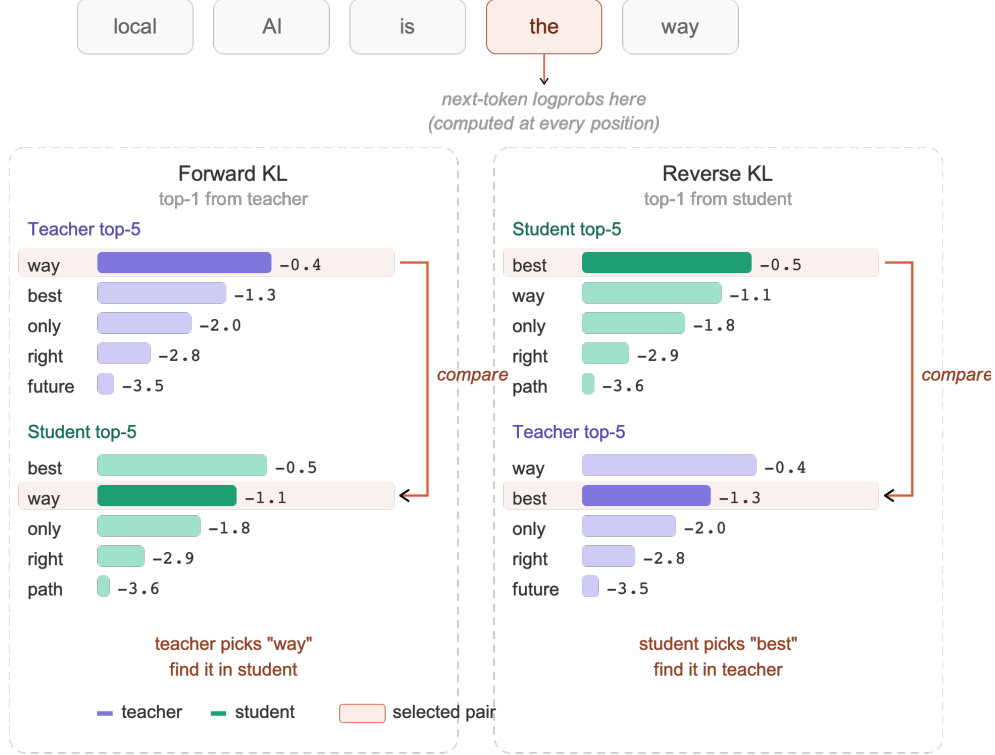


Figure 3.3: Top- K approximation of the forward and reverse KL. Instead of summing over the full vocabulary V , the K tokens with the highest teacher (forward) or student (reverse) probability are retained explicitly and the remaining vocabulary is grouped into a tail bucket.

For reasons explained in Section 4.3, we also use a variant that keeps the reverse-KL direction but selects the top- k support from the teacher. We call this loss the *Teacher-Support Reverse KL* (TS-RKL), because it uses the student-to-teacher KL ratio on the teacher-selected buckets:

$$\hat{D}_{\text{TS-RKL}}^k(\pi_\theta \| \pi_T)(y | x) = \sum_{n=1}^{L_y} \sum_{b \in S_{T,n}^k \cup \{\tau_{S_{T,n}^k}\}} \pi_\theta^{[S_{T,n}^k]}(b | h_n) \log \frac{\pi_\theta^{[S_{T,n}^k]}(b | h_n)}{\pi_T^{[S_{T,n}^k]}(b | h_n)}. \quad (3.19)$$

TS-RKL is not the standard top- k reverse KL in Equation 3.18, because the explicit token buckets are chosen by the teacher rather than the student. The objective is still reverse KL in direction: the log-ratio is $\log \pi_\theta - \log \pi_T$ and the expectation is over the student distribution on the teacher-induced buckets.

3.5 Metrics

Top- k Support Coverage. A teacher-support top- k reverse-KL objective can only provide token-level supervision on the tokens in the teacher’s top- k set. Student probability mass placed on tokens outside that set is therefore not supervised at the token level. We measure how much of the student’s next-token distribution is spanned by the teacher’s top- k support. Let $\mathcal{H}_{\text{eval}}$ be the set of prefixes used for evaluation, with each prefix written as $h_n = (x, y_{<n})$. Using the teacher top- k set $S_{T,n}^k$ from Equation 3.15, we define the top- k support coverage as

$$\text{Coverage}_k = \frac{1}{|\mathcal{H}_{\text{eval}}|} \sum_{h_n \in \mathcal{H}_{\text{eval}}} \sum_{v \in S_{T,n}^k} \pi_\theta(v | h_n). \quad (3.20)$$

This is the student probability mass that falls inside the teacher’s top- k , averaged over prefixes. Its complement $1 - \text{Coverage}_k$ is the student mass lying outside the teacher’s top- k , i.e. the fraction of the student’s distribution that a teacher top- k support cannot supervise at the token level.

Gap Recovery. We are not only interested in the final performance of the student, but also in how much of the teacher performance the student recovers. Following [18], we measure the recovered teacher-student gap as

$$\text{GapRecovery} = \frac{\text{Acc}_{\text{After}} - \text{Acc}_{\text{Before}}}{\text{Acc}_{\text{Teacher}} - \text{Acc}_{\text{Before}}}, \quad (3.21)$$

where $\text{Acc}_{\text{Before}}$ and $\text{Acc}_{\text{After}}$ are the student performance before and after training, respectively.

Chapter 4

Experimental Setup

4.1 Data

All training runs use prompts from DeepMath-103K [11], a large-scale mathematical reasoning dataset available on Hugging Face¹. This dataset is well matched to our target setting because it contains difficult, verifiable mathematics problems rather than general instruction-following prompts. The problems span a broad range of mathematical topics and were designed to be challenging and decontaminated from common evaluation benchmarks. We use the prompts as the common input distribution across all methods so that differences between runs can be attributed to the distillation objective and teacher choice rather than to differences in task selection.

For off-policy experiments that require precomputed teacher data rollouts, we generate the teacher completions with DataTrove [26] using the hyperparameters described in Appendix A.4. We make the generated datasets available on Hugging Face.

4.2 Models

We use the Qwen3 model family [44] to study different levels of the capacity gap. Keeping teacher and student within the same model family is important because architecture, tokenizer, training data, and chat-template differences can introduce stylistic and distributional mismatches that are unrelated to model size. Those mismatches can degrade distillation performance and obscure the specific effect of the teacher–student capacity gap [18]. Crucially, [44] report that the small models go through a distillation process from larger models in the Qwen3 family. This prior distillation makes it even more important to be consistent with the model family from the teacher previously used for distillation [41]. We include preliminary experiments with different model families in Appendix A.5, but the main experiments focus on Qwen3.

Specifically, we use Qwen3-0.6B and Qwen3-4B in non-thinking mode as students, and Qwen3-4B-Instruct, Qwen3-30B-A3B-Instruct-2507, and Qwen3-235B-A22B-Instruct-2507 as teachers. For the initial student models, we modify only the chat template, not the weights, to force non-thinking mode. This ensures that the student remains in the intended inference mode throughout training and preserves the baseline behavior reported by the Qwen team.

¹<https://huggingface.co/datasets/zwhe99/DeepMath-103K>

4.3 Inference Engine and Computational Resources

We use vLLM [17] to generate student rollouts and teacher log probabilities. vLLM varies significantly between versions, so we pin version 0.17.1 for all our experiments. Using 0.17.1 instead of the latest 0.20.0 means we can only get the top-k log-probabilities from a vLLM server but cannot get specific log-probabilities. As shown in Figure 3.3, we require specific log-probabilities from the teacher based on the top-k under the student to calculate the strict top-k RKL. That is why we need to define the alternative TS-RKL formulation explained in Section 3.4

The 4B teacher fits on the same GPUs used for training, while the 30B and 235B teachers are served through separate vLLM servers. Throughout the experiments, student training uses 4 nodes with 8 NVIDIA H100 GPUs each. The 30B teacher uses 1 additional serving node, and the 235B teacher uses 2 additional serving nodes. We include all the setting for rollout generation and server configuration in the Appendix A.4.

4.4 Benchmarks

Our target capability is mathematical reasoning, so AIME25 [49] is the main evaluation benchmark for the experiments. AIME is a competition-style high-school mathematics exam with short numeric answers. The problems require multi-step algebraic, geometric, combinatorial, and number-theoretic reasoning, and they are difficult to solve by pattern matching alone. This makes AIME25 a useful endpoint for measuring whether distillation improves the kind of structured mathematical reasoning targeted by DeepMath-103K.

We also evaluate on two auxiliary benchmarks to measure whether training on mathematics improves the target capability at the cost of broader behavior. GPQA [28] measures graduate-level scientific reasoning through multiple-choice questions written by domain experts in biology, physics, and chemistry. The questions are designed to be difficult for skilled non-experts even with web access, so performance on GPQA gives a signal about whether the model preserves harder out-of-domain reasoning skills. IFEval [51] measures instruction following using prompts with automatically verifiable constraints, such as required keywords, formatting rules, or length restrictions. We include it because reasoning fine-tuning and distillation can change a model’s compliance with explicit user instructions even when the training data is not directly about instruction following.

We run all evaluations with the Inspect library [2]. Using the same evaluation harness across checkpoints keeps prompting, sampling, answer extraction, and scoring consistent, which is especially important when comparing small changes between distillation objectives and teacher sizes.

4.5 Implementation

Distillation requires access to log probabilities from both the teacher and the student in order to compute the losses above. In OPD this requirement is coupled to rollout generation: the training examples are not fixed completions, but samples from the current student policy π_θ . For small teachers, the simplest implementation is to colocate the teacher, the student, and the rollout generator on the same training GPUs. This becomes impractical when the teacher is tens or hundreds of billions of parameters, because the training GPUs must already store the student, optimizer state, activations for backpropagation, and the inference engine used for student generation. We therefore separate the teacher from the training job and query it through an external server, following the large-teacher setup described in [24].

The on-policy part of the algorithm creates a systems constraint that is absent in ordinary supervised distillation. Rollouts cannot be generated once, cached, and reused throughout training, because they must come from the current student rather than from an older checkpoint. This means generation happens inside the training loop. In our implementation, vLLM is used as the inference engine for efficient student rollouts, while Transformers [40] is responsible for the student forward pass, log-probability computation, backpropagation, and optimizer updates. These two views of the student must be kept synchronized. After the optimizer changes the Transformers model weights, the vLLM copy used for generation must be refreshed before it is allowed to produce new on-policy samples. Otherwise the loss would be computed on trajectories from a stale policy, weakening the on-policy guarantee. A generation buffer can improve throughput by batching prompts across gradient accumulation steps, but only because the student weights remain fixed until the optimizer step.

Serving the teacher on separate GPUs solves the memory problem but turns teacher scoring into a distributed inference workload. In this setup, the *client* is the set of GPUs and processes used for training the student, and the *server* is the set of GPUs running the teacher model behind vLLM. The client sends the student-generated sequences to the server and receives the teacher log probabilities needed by the loss. This communication is expensive because log-probability responses are much larger than ordinary text-generation responses. Even with a top- k approximation, the server must return token ids and log probabilities for many positions in each sequence. This is especially challenging on the reasoning tasks we are interested in, where the number of tokens is large. The second challenge is latency. Multiple data-parallel training workers can query the same teacher server concurrently, and without careful request batching the server either underutilizes vLLM’s batching capacity or accumulates long queues that leave the training GPUs idle. For this reason, the teacher server must be engineered as part of the training system, with batching and compact serialization used to reduce both transfer cost and tail latency.

Our implementation addresses both of these challenges and enables distillation with large teachers. We published our implementation in the TRL library [38] as the `DistillationTrainer` class and provide a detailed benchmarking of our methods in a separate publication [24].

Chapter 5

Experiments

5.1 OPD with Privileged Context

Our first experiment tests a minimal privileged context that gives the teacher information about itself, the student, and the distillation task. This intervention is intentionally simple: it introduces the distinction between the student and teacher scoring contexts while providing a first diagnostic of whether a short PC can change downstream behavior. The PC we used is shown in Figure 5.1, where we compare the prompt seen by the student with the one seen by the teacher.

Despite being a PC intervention of less than 50 tokens, it already has a noticeable impact on downstream performance, as shown in Figure 5.2. The plots show that distillation does not benefit from using a larger teacher, regardless of whether it is off-policy (SFT), on-policy (OPD), or on-policy with privileged context (OPD + PC). Although the 235B teacher is more capable than the 4B and 30B teachers, each student reaches similar performance across teacher sizes. The plots also show that on-policy methods perform better than SFT.

Another conclusion from these plots is that informing the teacher about the distillation task through privileged context either degrades or maintains performance on the math reasoning task. This degradation is seen more clearly in Figure 5.3, where we show the gap recovery rate defined in Section 3.5. For the 0.6B student, the recovery rate declines 41% when scaling from a 4B to a 235B teacher. This decline is more pronounced than when using OPD without PC. A similar pattern emerges for the 4B student: OPD without PC has a smaller degradation when scaling from a 30B to a 235B teacher.

One difference between the two students is how SFT behaves when scaling the teacher. For the 0.6B student, SFT has the largest degradation in gap recovery rate, with a 50% decline. Conversely, for the 4B student, SFT has the smallest decline among the three methods. We do not attribute this pattern to a specific mechanism, but it reinforces that the effect of teacher scaling depends on both the student capacity and the training objective.

Overall, these experiments show that not all PC interventions are useful and that even a short privileged context can affect downstream performance.

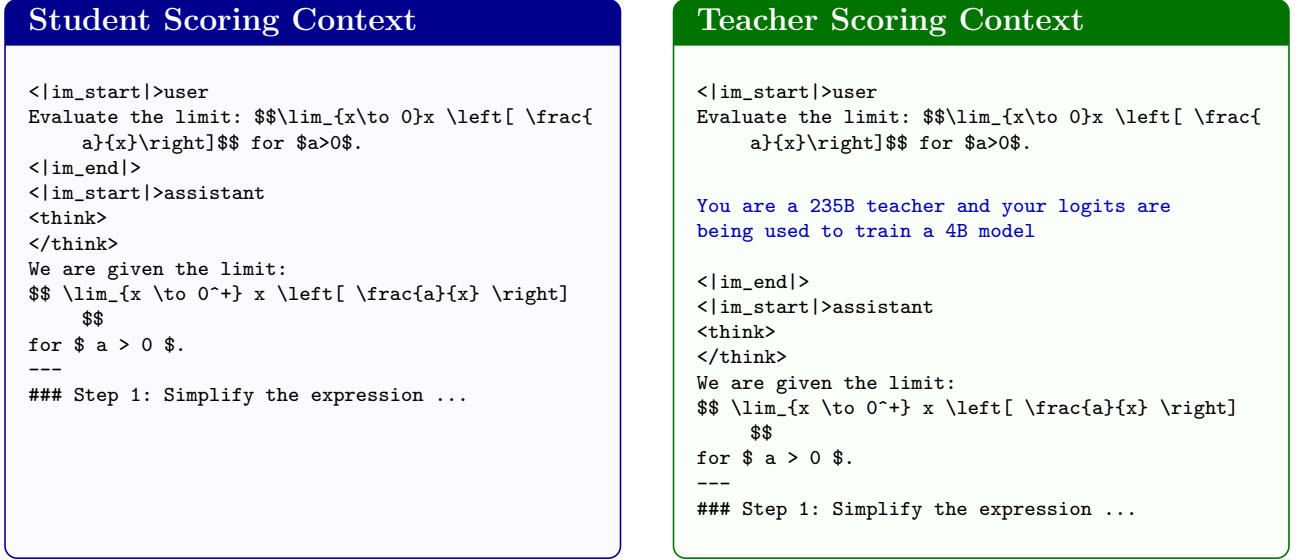


Figure 5.1: Example showing the difference between $\pi_\theta(\cdot \mid x, y_{<n})$ (Student Scoring Context) and $\pi_T(\cdot \mid x, c, y_{<n})$ (Teacher Scoring Context). The privileged context is *You are a 235B teacher and your logits are being used to train a 4B model*. We add it to the user turn seen by the teacher when generating the logits, and it is the only difference between the text scored by the student and teacher policies.

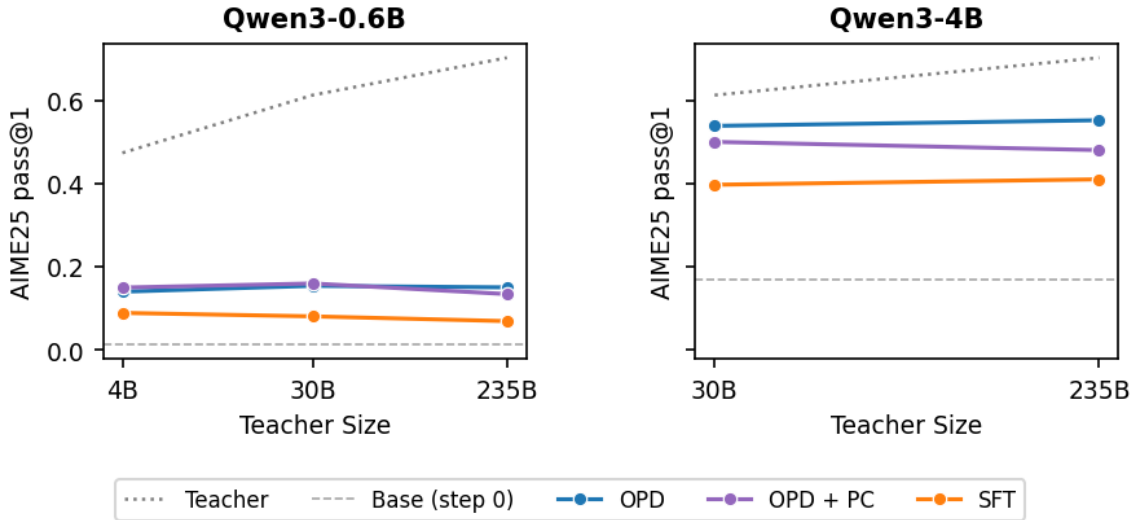


Figure 5.2: AIME scores comparing an off-policy distillation method (SFT) with OPD distillation methods. The plots show that using OPD results in better performance for both students regardless of teacher size. The effect of teacher scaling is easier to interpret through the gap recovery rate in Figure 5.3.

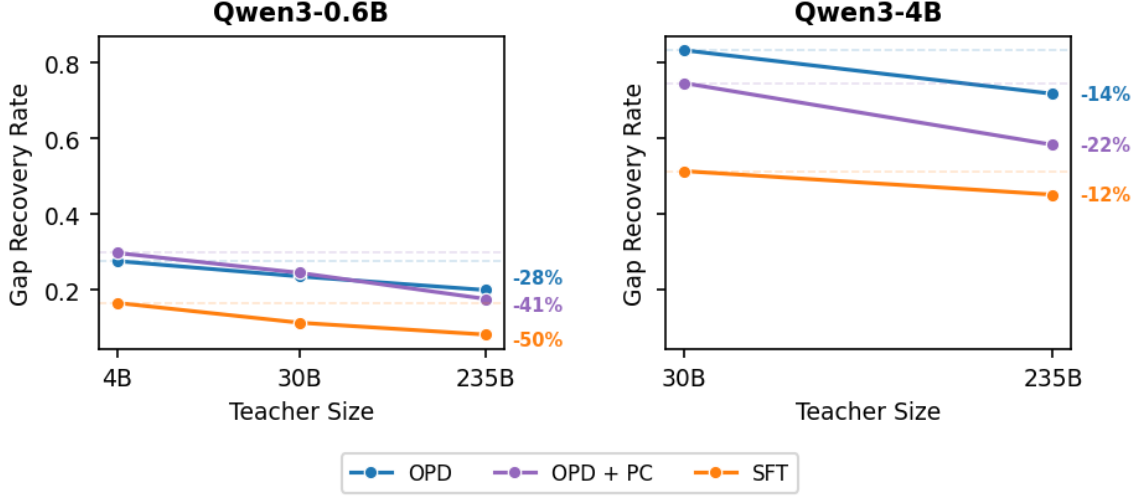


Figure 5.3: Gap recovery rate for all student-teacher combinations. The performance of the 0.6B student degrades as we increase the teacher size, and the decline is most pronounced for SFT, followed by OPD with privileged context. For the 4B student, privileged context also causes the steepest decline when scaling the teacher, but still performs better than SFT.

5.2 Context Distillation vs. Direct System Prompt

The simple PC that informed the teacher model that its logits were being used to train a small student degraded performance compared to using OPD without PC. We therefore tested whether a more targeted PC, similar to [47], could address the main failure modes of the results shown in Figures 5.2 and 5.3.

After an error analysis of the students from Section 5.1, we classified the errors into 5 categories:

- **Truncation:** Generation cut off by the `max_tokens` budget before the model produced an `ANSWER:` line.
- **Out of range:** Extracted a clean integer outside $[0, 999]$. AIME answers are always in this range, so this is almost certainly a setup or sign error.
- **Format fail:** Not truncated, but the AIME scorer could not extract a short integer from the response.
- **Off-by-small:** Extracted a clean integer within ± 5 of the target, so this error is typical of an arithmetic slip late in an otherwise correct setup.
- **Wrong Substantive:** Extracted a clean integer within $[0, 999]$ that is more than 5 away from the target. This indicates a substantive error in the reasoning process.

As shown by Table 5.1, the predominant errors are truncation and wrong substantive. The truncation issue is especially prevalent in the 0.6B student when scaling the teacher. This is a symptom of the student receiving feedback from a model with more complex reasoning patterns but failing to use those reasoning patterns to reach a solution.

To address the two main failure modes, we included the privileged context shown in Figure 5.4, which encourages concise and direct mathematical reasoning. For the succinctness guideline, we use a portion of the prompt developed by [30]. We exclude the section that refers to the format expected by AIME to focus on general math reasoning skills instead of formatting issues specific to a benchmark.

Table 5.1: Share of incorrect attempts (% of incorrect) per error category in AIME25.

Student / Teacher	Truncation	Wrong substantive	Off-by-small	Out of range	Format fail
4B / 30B	44%	49%	4%	1%	2%
4B / 235B	54%	42%	3%	1%	0%
0.6B / 4B	41%	46%	3%	7%	3%
0.6B / 30B	87%	9%	2%	1%	0%
0.6B / 235B	86%	11%	1%	2%	0%

Privileged Context (Teacher)	System Prompt (Teacher & Student)
<pre> < im_start >user Evaluate the limit: $\lim_{x \rightarrow 0} x \left[\frac{a}{x} \right]$ for $a > 0$. Solve the following math problem concisely and correctly. Be direct; avoid unnecessary elaboration, redundant steps, or restating the problem. Focus only on the key reasoning steps needed to reach the answer. < im_end > < im_start >assistant <think> </think> We are given the limit: $\lim_{x \rightarrow 0^+} x \left[\frac{a}{x} \right]$ for $a > 0$. --- ### Step 1: Simplify the expression ... </pre>	<pre> < im_start >system Solve the following math problem concisely and correctly. Be direct avoid unnecessary elaboration, redundant steps, or restating the problem. Focus only on the key reasoning steps needed to reach the answer. However, don't omit verification and prioritize correctness over brevity. < im_end > < im_start >user Evaluate the limit: $\lim_{x \rightarrow 0} x \left[\frac{a}{x} \right]$ for $a > 0$. < im_end > < im_start >assistant <think> </think> We are given the limit: $\lim_{x \rightarrow 0^+} x \left[\frac{a}{x} \right]$ for $a > 0$. --- ### Step 1: Simplify the expression ... </pre>

Figure 5.4: Comparison between providing the targeted intervention as privileged context or through a system prompt. The key difference is that the privileged context is only seen by the teacher while the system prompt is seen by the teacher and the student.

Besides using the privileged context, we also show in Figure 5.4 how we introduce the same instruction through the system prompt. This lets us compare two ways of steering student behavior in an OPD task: placing the instruction only in the teacher’s scoring context, or placing it in the shared context used by both the student and the teacher. The results in Figure 5.5 compare these interventions against OPD training runs without the brevity instruction. The system prompt is more effective than using PC to distill the instruction into the student.

These results align with the advantage of matching the student’s rollout distribution to the intended behavior. With the system prompt, the student generates rollouts while already conditioned to be concise, so it visits states that follow the instruction and receives teacher feedback on those states. With privileged context, the student generates rollouts without the brevity instruction; only the teacher sees it when scoring the student’s tokens. The teacher feedback may therefore reflect an instruction that the student did not use to produce the rollout, and the student may not have enough capacity to infer the intended behavior from that indirect signal. This helps explain why using the system prompt directly on the student achieves equal or better performance than OPD and OPD + PC.

5.3 Rollout Lengths and Epistemic Words

Besides the AIME scores, we tracked the mean rollout length during training and the length of the responses when solving AIME problems to determine how well PC and the system prompt steered the student model toward shorter rollouts. We show the results in Figure 5.6, where we can see the effect of scaling the teacher on student rollout length. The student rollout

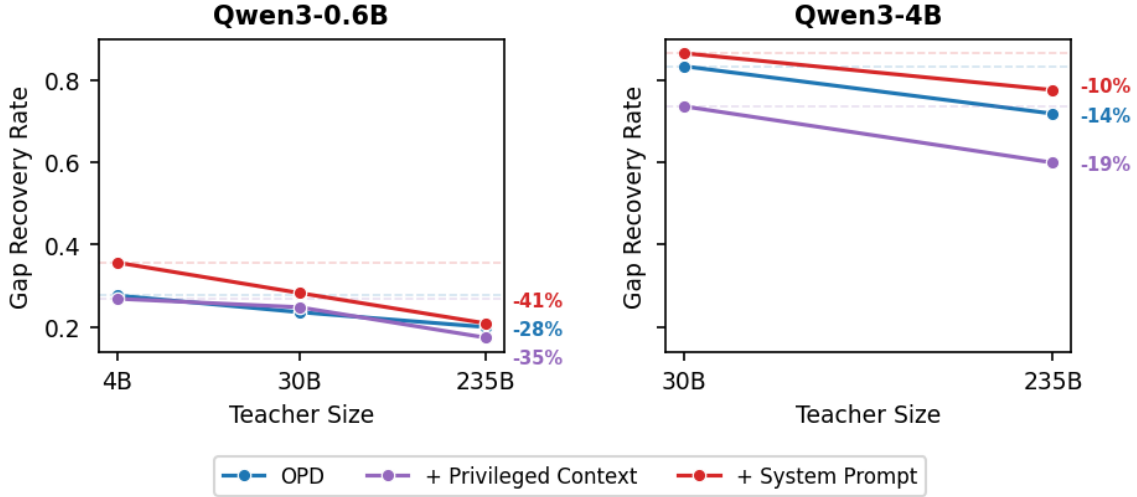


Figure 5.5: Plot comparing the performance of a privileged context with passing the same information through the system prompt to encourage brevity in the model response. The plots show that using the system prompt instead of the privileged information is more effective in terms of AIME performance.

length increases up to 24x for the largest teacher compared to the mean length before training. As we increase the teacher size, the student rollouts also increase in length, suggesting that the student is using longer traces to absorb feedback from the teacher. This behavior is more pronounced in the 0.6B student, which shows how the capacity gap manifests in rollout length. In other words, the 4B student has a larger capacity than the 0.6B student, so it does not need to increase its rollout length as much to accommodate the information passed by larger teachers.

One possible explanation for the longer student rollouts is that larger teachers naturally produce longer solutions, which the students then imitate during distillation. However, the teacher rollout statistics in Table 5.2 do not support this explanation. The 4B, 30B, and 235B teachers have broadly similar rollout length distributions: all three reach the same p99 truncation point, their medians differ by at most 105 tokens, and their means differ by about 122 tokens. The median is also not monotonic with teacher size, since the 4B teacher has a longer median rollout than both the 30B and 235B teachers. These differences are small compared to the student rollout increases in Figure 5.6. This suggests that the longer student rollouts are better understood as a coping mechanism for absorbing more complex teacher feedback, rather than as direct imitation of naturally longer teacher completions.

Table 5.2: Teacher rollout length statistics for the 4B, 30B, and 235B teachers.

Statistic	4B teacher	30B teacher	235B teacher
Max	8,195	8,192	8,193
p99	8,192	8,192	8,192
p95	7,280	7,756	7,854
Median	1,714	1,609	1,667
Mean	2,517.9	2,534.3	2,639.9
Min	1	1	1

The increase in rollout length is interesting because the students we train are hybrid models. Although they are trained without reasoning enabled and receive feedback from instruct teachers, they have the latent ability to activate reasoning patterns when necessary. The increase in

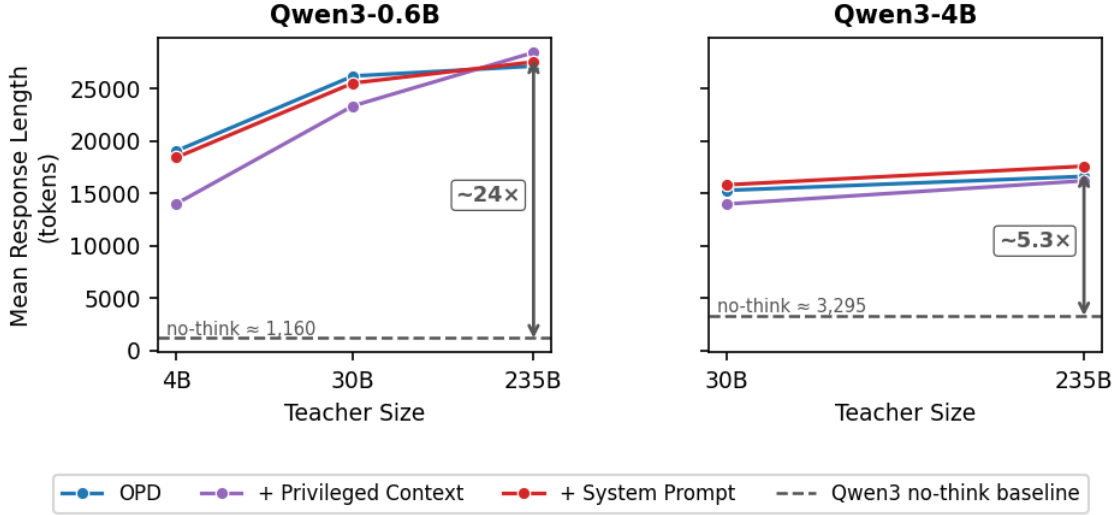


Figure 5.6: Plots showing how the mean response length increases for both students after OPD. The figure also shows that scaling the teacher size causes the rollout length to increase, and that this scaling is more pronounced for the 0.6B student.

rollout length may be a symptom of this reasoning mode taking over in the student model, but a more direct signal is the inclusion of epistemic words in the response. We study the number of epistemic words as defined by [16] and show our results in Figure 5.7. The results show an increase of 13–23 times in the number of epistemic words per rollout compared to the 0.6B student model before distillation. We also see an increase of 3.5–5 times in the 4B student.

Despite the growth in epistemic words compared to the initial student model, we do not see the same scaling pattern that we observed for rollout length in Figure 5.6. Instead, the number of epistemic words remains roughly constant for the PC and system-prompt training runs that ask the student model to be succinct. This lack of correlation suggests that longer rollouts are not simply explained by more epistemic self-talk. Rather, teacher scaling appears to increase trace length through a mechanism that is distinct from the frequency of these reasoning markers.

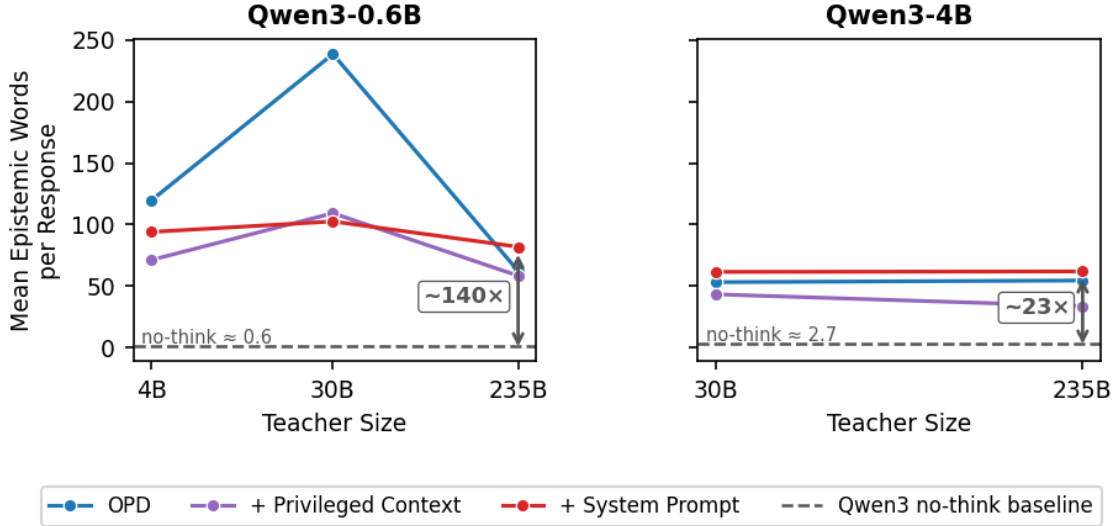


Figure 5.7: Number of epistemic words per rollout for different types of OPD setups. The plots show that all methods increase the number of epistemic words per rollout for all teacher sizes, but that the behavior does not scale with teacher size when we instruct the student model to be succinct in its answers.

5.4 Using Privileged Context for Teacher-Assisted Distillation

One of the first works that addressed the capacity gap proposed progressively distilling from larger teachers until reaching a teacher scale that was accessible to the student [22]. For example, in our case we could distill the 235B model into the 30B model, then that 30B model into the 4B model, and finally distill that 4B model into our 0.6B student. However, that process would be expensive because it would require three distillation runs to reach the objective.

We wanted to follow the same idea in the context of OPD and PC while avoiding multiple distillation runs. We set up the distillation with the 0.6B student and the 4B teacher, as this combination has a small capacity gap and performance comparable to other student-teacher combinations. Instead of incorporating instructions or information about the problem as in Figures 5.1 and 5.4, we include a rollout from the 30B or 235B teacher for that prompt in the 4B teacher’s privileged context. The larger teacher’s rollout is therefore visible only to the 4B teacher when it scores the student’s tokens, while the 0.6B student still generates rollouts from its own context. That way, the 4B teacher effectively acts as a bridge between the larger teachers and the 0.6B student. We follow an approach similar to [31], where they incorporate demonstrations from GPT-4o to show the student how the problem can be solved.

5.5 Extending from top-1 to top- K Log Probabilities

Because PC and system-prompt interventions did not solve the capacity gap when scaling the teacher, we next explored results showing that increasing the number of log probabilities used for training can improve student performance in OPD settings [14]. Scaling the number of log probabilities is expensive when using a separate server for the teacher, because we need to transfer the log probabilities from the server hosting the teacher to the GPUs hosting the student model [24]. For that reason, we restrict our analysis to $K \in \{1, 10, 50\}$.

We calculated the probability mass coverage, as defined in Section 3.5, for each K value and student-teacher combination. The results in Table 5.3 show that mass coverage increases

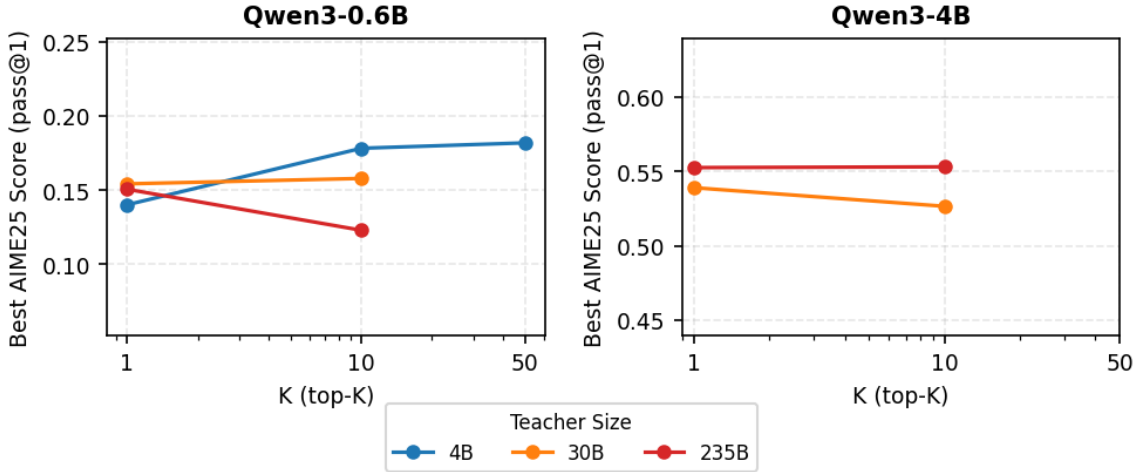


Figure 5.8: Effect of increasing the top- K log probabilities. The 0.6B student benefits from increasing K for the smallest teacher, but its performance degrades when using more log probabilities from the 235B teacher. These results suggest that the 0.6B student cannot capture the increasing complexity of the large teacher but benefits from more detailed feedback from a teacher within its reach. For the 4B student, scaling K does not have a positive impact since its performance remains constant or decreases for all teachers.

quickly with K . With $K = 10$, coverage is already above 0.92 for every evaluated pair, and with $K = 50$ it reaches at least 0.986. For the larger teachers, $K = 50$ reaches 0.993 coverage for the 0.6B student and at least 0.997 coverage for the 4B student. This suggested that increasing K might improve student performance because the loss captures most of the teacher probability mass.

Table 5.3: Probability mass coverage for different top- K values and student-teacher combinations. The 4B/4B setting was not evaluated.

Student	Teacher	$K = 1$	$K = 10$	$K = 50$
0.6B	4B	0.702	0.923	0.986
0.6B	30B	0.713	0.941	0.993
0.6B	235B	0.713	0.941	0.993
4B	4B	—	—	—
4B	30B	0.790	0.972	0.997
4B	235B	0.775	0.974	0.998

However, Figure 5.8 contradicts the hypothesis that higher coverage should directly improve student performance. AIME scores decrease or remain stable as we increase K . The only exception is the 0.6B student trained with the 4B teacher, where performance improves as we scale K . This contrast is especially clear for the larger teachers: even when $K = 50$ covers 0.993–0.998 of the probability mass, performance does not improve.

This behavior presents another symptom of the capacity gap and shows how a small student struggles to use feedback from a much larger teacher. In all cases, performance either stays the same or decreases when using the 30B and 235B teachers. This behavior shows that larger probability coverage does not necessarily translate into higher student performance. Coverage measures how much of the teacher distribution is exposed to the student, but not whether the student can represent or learn from that distribution. Additionally, it highlights the challenge of giving effective feedback in scenarios with a large capacity gap.

Chapter 6

Conclusions

This thesis set out to study knowledge distillation under extreme teacher–student capacity gaps. The motivation was straightforward: the most useful deployment setting for distillation is often the hardest one. We would like to train very small students because they are cheap and fast to serve, but the most attractive teachers are much larger models because they are more capable. The central question was therefore whether on-policy distillation (OPD), privileged context (PC), and richer token-level teacher feedback could make supervision from very large teachers more usable for much smaller students.

The original hypothesis was that OPD and PC would complement each other. OPD should help because the teacher scores states visited by the student rather than states sampled from a fixed teacher-generated dataset. PC should help because the larger teacher can use additional hidden context when producing its feedback, while the student remains small and unconditioned at inference time. In principle, this combination could exploit the teacher’s stronger in-context learning ability without increasing the student’s inference cost. We also intended to study whether making rollouts shorter, increasing the number of teacher log probabilities, and using larger teachers would improve the efficiency and performance of the distilled student.

The experiments support the first part of this motivation but not the stronger version of the hypothesis. OPD is consistently better than off-policy SFT in the mathematical reasoning setting we study, which confirms that training on student-generated rollouts is useful under large capacity gaps. However, OPD does not by itself make the largest teacher the best teacher. Scaling from smaller teachers to the 235B teacher does not reliably improve AIME performance, and in several settings the gap recovery rate decreases as the teacher becomes larger. This is the main empirical result of the thesis: under extreme capacity gaps, a more capable teacher does not automatically provide a more learnable training signal.

Privileged context also turned out to be more fragile than expected. A simple PC that tells the teacher about the student and the distillation task already changes downstream performance, but it does not improve the capacity-gap problem and can make teacher scaling worse. A more targeted PC designed to encourage concise mathematical reasoning also fails to beat a simpler alternative: placing the same instruction directly in the system prompt seen by the student. This distinction matters because, in OPD, the student determines the rollout distribution. If the instruction is only hidden in the teacher’s scoring context, the student does not generate its rollout under the desired behavior. Direct system prompting changes the states the student visits, and the teacher then provides feedback on those states. In our experiments, this direct conditioning is more effective than trying to distill the hidden context indirectly through teacher logits.

The failure analysis shows why this matters. Truncation and substantive reasoning errors dominate the incorrect AIME attempts, with truncation becoming especially severe for the 0.6B student trained with larger teachers. Teacher scaling also increases rollout length substantially,

most strongly for the smallest student. This suggests that small students cope with large-teacher feedback by producing longer traces, but the epistemic-word analysis shows that length growth is not simply the same as more explicit reasoning markers. OPD increases epistemic wording relative to the base students, but those markers do not scale with teacher size in the same way rollout length does under brevity interventions. The length explosion therefore appears to be a broader symptom of the capacity gap rather than merely an increase in epistemic self-talk.

The top- K experiments reinforce the same conclusion from another direction. Increasing K improves probability mass coverage and should, in principle, give the student a richer approximation of the teacher distribution. In practice, this extra information does not reliably improve performance. The 0.6B student benefits from larger K only when the teacher is the nearby 4B model. With larger teachers, and for the 4B student, increasing K leaves performance unchanged or makes it worse. Higher coverage is therefore not enough: the additional probability mass must also be usable by the student. Under a large capacity gap, richer feedback can become harder rather than easier to learn from.

Taken together, these results shift the conclusion from “how can we get more signal from the largest teacher?” to “how can we make the teacher signal match the student’s capacity and rollout distribution?” The experiments suggest that interventions which directly shape the student’s behavior are more reliable than interventions visible only to the teacher. They also show that teacher scale, probability mass coverage, and hidden context are not sufficient proxies for useful supervision. A teacher signal can be more capable, more detailed, and more informed while still being less learnable for the student.

There are also clear limitations. The teacher-assisted privileged-context experiment is still a setup rather than a completed result, so we cannot yet say whether using a larger teacher’s rollout as hidden context for a smaller bridge teacher solves the capacity-gap problem. The top- K analysis is limited to $K \in \{1, 10, 50\}$ because transferring teacher log probabilities is expensive at this scale. The experiments also focus on Qwen3 models and mathematical reasoning, so the conclusions should be tested on other model families and tasks before treating them as universal properties of OPD.

Despite these limits, the thesis provides a practical picture of extreme-capacity-gap distillation. OPD is a strong baseline and a better starting point than off-policy SFT, but it does not remove the need to match teacher feedback to student capacity. Privileged context is useful as an experimental tool, but not a general solution by itself. Directly conditioning the student can be more effective than hiding instructions in the teacher. Finally, scaling the amount of teacher information, whether through larger teachers or more top- K log probabilities, can hurt when the student cannot absorb the added complexity. The systems contribution of the thesis makes these questions easier to study by providing an efficient OPD trainer for large remote teachers, but the empirical lesson is that future methods should focus less on maximizing teacher strength and more on making teacher feedback accessible to the student.

Appendix A

Appendix

A.1 OPD vs Off-Policy Methods

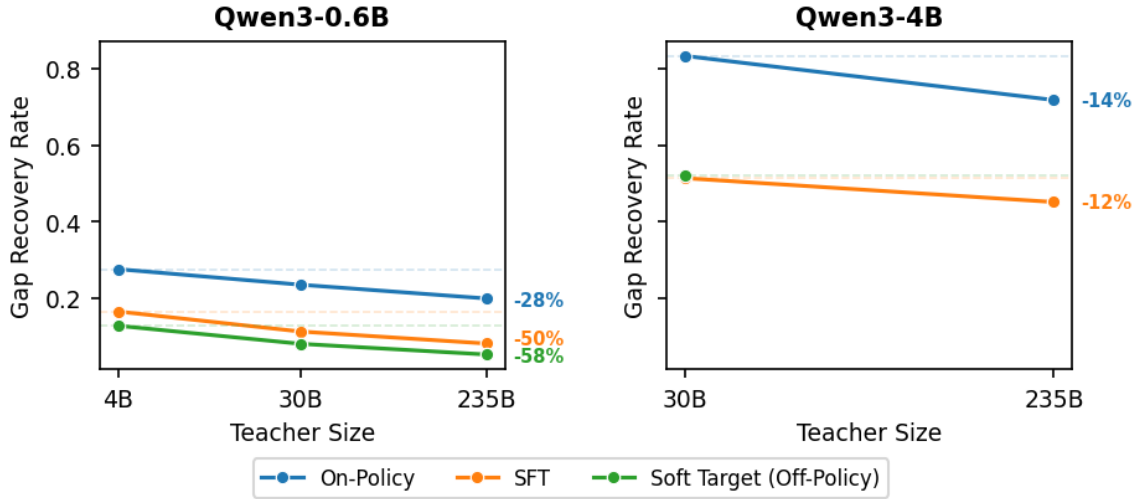


Figure A.1: Effect of increasing the top-k log probabilities. The 0.6B benefits from increasing K for the smallest teacher, but its performance degrades when using more logprobs from the 235B teacher. These results suggest that the 0.6B student cannot capture the increasing complexity of the large teacher but benefits from more details from a teacher within its reach. For the 4B student, scaling K does not have a positive impact since its performance remains constant or decreases for all teachers.

A.2 Catastrophic Forgetting

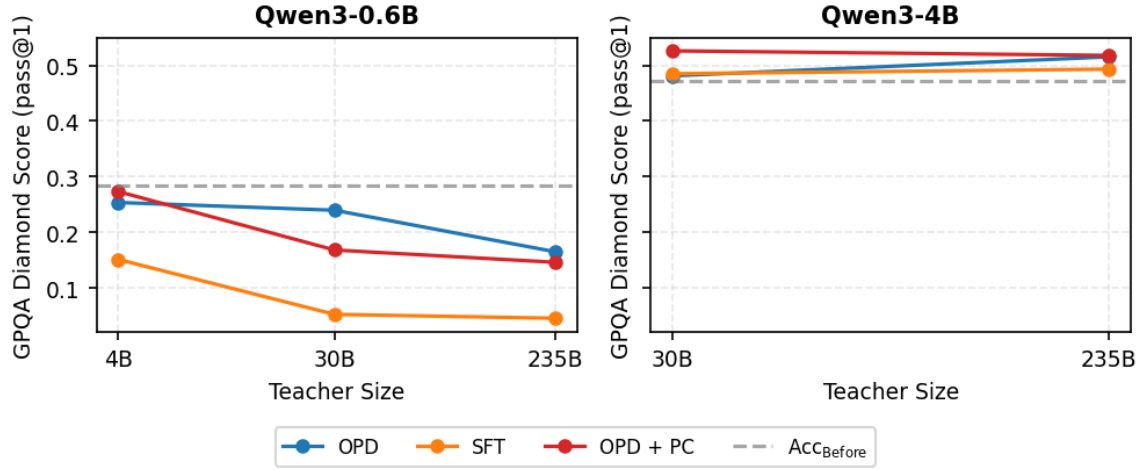


Figure A.2: Scores for GPQA for all students while training targeting math reasoning skills. The plots show how the 0.6B student suffers from catastrophic forgetting for the three methods, but also how SFT is the method that degrades performance the most. For the 4B student, all methods preserve the GPQA performance and even marginally improve it. These results show how the main driver in catastrophic forgetting is model capacity, and that on-policy methods are better to mitigate performance degradation.

A.3 TAID Results

A.4 Hyperparameters

A.5 Training across Families

A.6 Privileged Information Prompts

Bibliography

- [1] Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes, 2024.
- [2] UK AI Security Institute. Inspect AI: Framework for Large Language Model Evaluations, May 2024.
- [3] Mohammadreza Armandpour, Fatih Ilhan, David Harrison, Ajay Jaiswal, Duc N. M. Hoang, Fartash Faghri, Yizhe Zhang, Minsik Cho, and Mehrdad Farajtabar. Unmasking On-Policy Distillation: Where It Helps, Where It Hurts, and Why, May 2026.
- [4] Amanda Askell, Yuntao Bai, Anna Chen, Dawn Drain, Deep Ganguli, Tom Henighan, Andy Jones, Nicholas Joseph, Ben Mann, Nova DasSarma, Nelson Elhage, Zac Hatfield-Dodds, Danny Hernandez, Jackson Kernion, Kamal Ndousse, Catherine Olsson, Dario Amodei, Tom Brown, Jack Clark, Sam McCandlish, Chris Olah, and Jared Kaplan. A General Language Assistant as a Laboratory for Alignment, December 2021.
- [5] Aman Bhargava, Cameron Witkowski, Alexander Detkov, and Matt Thomson. Prompt Baking, September 2024.
- [6] Nicolas Boizard, Kevin El Haddad, Céline Hudelot, and Pierre Colombo. Towards Cross-Tokenizer Distillation: The Universal Logit Distillation Loss for LLMs, January 2025.
- [7] Howard Chen, Noam Razin, Karthik Narasimhan, and Danqi Chen. Retaining by Doing: The Role of On-Policy Data in Mitigating Forgetting, December 2025.
- [8] DeepSeek-AI. DeepSeek-V4: Towards highly efficient million-token context intelligence, 2026.
- [9] Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. MiniLLM: On-Policy Distillation of Large Language Models, January 2026.
- [10] Yinghui He, Simran Kaur, Adithya Bhaskar, Yongjin Yang, Jiarui Liu, Narutatsu Ri, Liam Fowl, Abhishek Panigrahi, Danqi Chen, and Sanjeev Arora. Self-Distillation Zero: Self-Revision Turns Binary Rewards into Dense Supervision, April 2026.
- [11] Zhiwei He, Tian Liang, Jiahao Xu, Qiuzhi Liu, Xingyu Chen, Yue Wang, Linfeng Song, Dian Yu, Zhenwen Liang, Wenxuan Wang, Zhuosheng Zhang, Rui Wang, Zhaopeng Tu, Haitao Mi, and Dong Yu. DeepMath-103K: A Large-Scale, Challenging, Decontaminated, and Verifiable Mathematical Dataset for Advancing Reasoning, May 2025.
- [12] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the Knowledge in a Neural Network, March 2015.

- [13] Haiduo Huang, Jiangcheng Song, Yadong Zhang, and Pengju Ren. SelecTKD: Selective Token-Weighted Knowledge Distillation for LLMs, November 2025.
- [14] Jonas Hübotter, Frederike Lübeck, Lejs Behric, Anton Baumann, Marco Bagatella, Daniel Marta, Ido Hakimi, Idan Shenfeld, Thomas Kleine Buening, Carlos Guestrin, and Andreas Krause. Reinforcement Learning via Self-Distillation, January 2026.
- [15] Ijun Jang, Jewon Yeom, Juan Yeo, Hyunggu Lim, and Taesup Kim. Stable On-Policy Distillation through Adaptive Target Reformulation, 2026.
- [16] Jeonghye Kim, Xufang Luo, Minbeom Kim, Sangmook Lee, Dohyung Kim, Jiwon Jeon, Dongsheng Li, and Yuqing Yang. Why Does Self-Distillation (Sometimes) Degrade the Reasoning Capability of LLMs?, March 2026.
- [17] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.
- [18] Yaxuan Li, Yuxin Zuo, Bingxiang He, Jinqian Zhang, Chaojun Xiao, Cheng Qian, Tianyu Yu, Huan-ang Gao, Wenkai Yang, Zhiyuan Liu, and Ning Ding. Rethinking On-Policy Distillation of Large Language Models: Phenomenology, Mechanism, and Recipe, April 2026.
- [19] Yuetai Li, Xiang Yue, Zhangchen Xu, Fengqing Jiang, Luyao Niu, Bill Yuchen Lin, Bhaskar Ramasubramanian, and Radha Poovendran. Small Models Struggle to Learn from Strong Reasoners, November 2025.
- [20] Kevin Lu and Thinking Machines Lab. On-Policy Distillation. *Thinking Machines Lab: Connectionism*, 2025.
- [21] Benjamin Minixhofer, Ivan Vulić, and Edoardo Maria Ponti. Universal Cross-Tokenizer Distillation via Approximate Likelihood Matching, October 2025.
- [22] Seyed-Iman Mirzadeh, Mehrdad Farajtabar, Ang Li, Nir Levine, Akihiro Matsukawa, and Hassan Ghasemzadeh. Improved Knowledge Distillation via Teacher Assistant, December 2019.
- [23] Shankar Padmanabhan, Yasumasa Onoe, Michael J. Q. Zhang, Greg Durrett, and Eunsol Choi. Propagating Knowledge Updates to LMs Through Distillation, October 2023.
- [24] Carlos Miguel Patiño, Kashif Rasul, Edward Beeching, and Lewis Tunstall. Distilling 100B+ models 40x faster with TRL, 2026.
- [25] Emiliano Penaloza, Dheeraj Vattikonda, Nicolas Gontier, Alexandre Lacoste, Laurent Charlin, and Massimo Caccia. Privileged Information Distillation for Language Models, February 2026.
- [26] Guilherme Penedo, Hynek Kydlíček, Alessandro Cappelli, Mario Sasko, and Thomas Wolf. DataTrove: Large scale data processing. GitHub, 2024.
- [27] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving Language Understanding by Generative Pre-Training. *2018*.

- [28] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. GPQA: A Graduate-Level Google-Proof Q&A Benchmark, November 2023.
- [29] Stephane Ross and Drew Bagnell. Efficient reductions for imitation learning. In Yee Whye Teh and Mike Titterton, editors, *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, volume 9 of *Proceedings of Machine Learning Research*, pages 661–668, Chia Laguna Resort, Sardinia, Italy, May 2010. PMLR.
- [30] Hejian Sang, Yuanda Xu, Zhengze Zhou, Ran He, Zhipeng Wang, and Jiachen Sun. CRISP: Compressed Reasoning via Iterative Self-Policy Distillation, April 2026.
- [31] Idan Shenfeld, Mehul Damani, Jonas Hübötter, and Pulkit Agrawal. Self-Distillation Enables Continual Learning, January 2026.
- [32] Wenxian Shi, Yuxuan Song, Hao Zhou, Bohan Li, and Lei Li. Follow Your Path: A Progressive Method for Knowledge Distillation. In Nuria Oliver, Fernando Pérez-Cruz, Stefan Kramer, Jesse Read, and Jose A. Lozano, editors, *Machine Learning and Knowledge Discovery in Databases. Research Track*, volume 12977, pages 596–611. Springer International Publishing, Cham, 2021.
- [33] Haebin Shin, Lei Ji, Yeyun Gong, Sungdong Kim, Eunbi Choi, and Minjoon Seo. Generative Prompt Internalization. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 7338–7363, Albuquerque, New Mexico, 2025. Association for Computational Linguistics.
- [34] Makoto Shing, Kou Misaki, Han Bao, Sho Yokoi, and Takuya Akiba. TAID: Temporally Adaptive Interpolated Distillation for Efficient Knowledge Transfer in Language Models, February 2025.
- [35] Charlie Snell, Dan Klein, and Ruiqi Zhong. Learning by Distilling Context, September 2022.
- [36] Shicheng Tan, Weng Lam Tam, Yuanchun Wang, Wenwen Gong, Yang Yang, Hongyin Tang, Keqing He, Jiahao Liu, Jingang Wang, Shu Zhao, Peng Zhang, and Jie Tang. GKD: A General Knowledge Distillation Framework for Large-scale Pre-trained Language Model, 2023.
- [37] Kimi Team, Yifan Bai, Yiping Bao, Y. Charles, Cheng Chen, Guanduo Chen, Haiting Chen, Huarong Chen, Jiahao Chen, Ningxin Chen, Ruijue Chen, Yanru Chen, Yuankun Chen, Yutian Chen, Zhuofu Chen, Jialei Cui, Hao Ding, Mengnan Dong, Angang Du, Chenzhuang Du, Dikang Du, Yulun Du, Yu Fan, Yichen Feng, Kelin Fu, Bofei Gao, Chenxiao Gao, Hongcheng Gao, Peizhong Gao, Tong Gao, Yuyao Ge, Shangyi Geng, Qizheng Gu, Xinran Gu, Longyu Guan, Haiqing Guo, Jianhang Guo, Xiaoru Hao, Tianhong He, Weiran He, Wenyang He, Yunjia He, Chao Hong, Hao Hu, Yangyang Hu, Zhenxing Hu, Weixiao Huang, Zhiqi Huang, Zihao Huang, Tao Jiang, Zhejun Jiang, Xinyi Jin, Yongsheng Kang, Guokun Lai, Cheng Li, Fang Li, Haoyang Li, Ming Li, Wentao Li, Yang Li, Yanhao Li, Yiwei Li, Zhaowei Li, Zheming Li, Hongzhan Lin, Xiaohan Lin, Zongyu Lin, Chengyin Liu, Chenyu Liu, Hongzhang Liu, Jingyuan Liu, Junqi Liu, Liang Liu, Shaowei Liu, T. Y. Liu, Tianwei Liu, Weizhou Liu, Yangyang Liu, Yibo Liu, Yiping Liu, Yue Liu, Zhengying Liu, Enzhe Lu, Haoyu Lu, Lijun Lu, Yashuo Luo, Shengling Ma, Xinyu Ma, Yingwei Ma, Shaoguang Mao, Jie Mei, Xin Men, Yibo Miao, Siyuan Pan, Yebo

Peng, Ruoyu Qin, Zeyu Qin, Bowen Qu, Zeyu Shang, Lidong Shi, Shengyuan Shi, Feifan Song, Jianlin Su, Zhengyuan Su, Lin Sui, Xinjie Sun, Flood Sung, Yunpeng Tai, Heyi Tang, Jiawen Tao, Qifeng Teng, Chaoran Tian, Chensi Wang, Dinglu Wang, Feng Wang, Hailong Wang, Haiming Wang, Jianzhou Wang, Jiaxing Wang, Jinhong Wang, Shengjie Wang, Shuyi Wang, Si Wang, Xinyuan Wang, Yao Wang, Yejie Wang, Yiqin Wang, Yuxin Wang, Yuzhi Wang, Zhaoji Wang, Zhengtao Wang, Zhengtao Wang, Zhexu Wang, Chu Wei, Qianqian Wei, Haoning Wu, Wenhao Wu, Xingzhe Wu, Yuxin Wu, Chenjun Xiao, Jin Xie, Xiaotong Xie, Weimin Xiong, Boyu Xu, Jinjing Xu, L. H. Xu, Lin Xu, Suting Xu, Weixin Xu, Xinran Xu, Yangchuan Xu, Ziyao Xu, Jing Xu, Jing Xu, Junjie Yan, Yuzi Yan, Hao Yang, Xiaofei Yang, Yi Yang, Ying Yang, Zhen Yang, Zhilin Yang, Zonghan Yang, Haotian Yao, Xingcheng Yao, Wenjie Ye, Zhuorui Ye, Bohong Yin, Longhui Yu, Enming Yuan, Hongbang Yuan, Mengjie Yuan, Siyu Yuan, Haobing Zhan, Dehao Zhang, Hao Zhang, Wanlu Zhang, Xiaobin Zhang, Yadong Zhang, Yangkun Zhang, Yichi Zhang, Yizhi Zhang, Yongting Zhang, Yu Zhang, Yutao Zhang, Yutong Zhang, Zheng Zhang, Haotian Zhao, Yikai Zhao, Zijia Zhao, Huabin Zheng, Shaojie Zheng, Longguang Zhong, Jianren Zhou, Xinyu Zhou, Zaida Zhou, Jinguo Zhu, Zhen Zhu, Weiyu Zhuang, and Xinxing Zu. Kimi K2: Open Agentic Intelligence, February 2026.

- [38] Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Gallouédec. TRL: Transformers reinforcement learning, 2020.
- [39] Shuohuan Wang, Yu Sun, Yang Xiang, Zhihua Wu, Siyu Ding, Weibao Gong, Shikun Feng, Junyuan Shang, Yanbin Zhao, Chao Pang, Jiaxiang Liu, Xuyi Chen, Yuxiang Lu, Weixin Liu, Xi Wang, Yangfan Bai, Qiuliang Chen, Li Zhao, Shiyong Li, Peng Sun, Dianhai Yu, Yanjun Ma, Hao Tian, Hua Wu, Tian Wu, Wei Zeng, Ge Li, Wen Gao, and Haifeng Wang. ERNIE 3.0 Titan: Exploring Larger-scale Knowledge Enhanced Pre-training for Language Understanding and Generation, December 2021.
- [40] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online, October 2020. Association for Computational Linguistics.
- [41] Yecheng Wu, Song Han, and Hai Cai. Lightning OPD: Efficient Post-Training for Large Reasoning Models with Offline On-Policy Distillation, April 2026.
- [42] Wenda Xu, Rujun Han, Zifeng Wang, Long T. Le, Dhruv Madeka, Lei Li, William Yang Wang, Rishabh Agarwal, Chen-Yu Lee, and Tomas Pfister. Speculative Knowledge Distillation: Bridging the Teacher-Student Gap Through Interleaved Sampling, 2024.
- [43] Shaotian Yan, Kaiyuan Liu, Chen Shen, Bing Wang, Sinan Fan, Jun Zhang, Yue Wu, Zheng Wang, and Jieping Ye. Distribution-Aligned Sequence Distillation for Superior Long-CoT Reasoning, January 2026.
- [44] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li,

- Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 Technical Report, May 2025.
- [45] Chenxu Yang, Chuanyu Qin, Qingyi Si, Minghui Chen, Naibin Gu, Dingyu Yao, Zheng Lin, Weiping Wang, Jiaqi Wang, and Nan Duan. Self-Distilled RLVR. <https://arxiv.org/abs/2604.03128v1>, April 2026.
 - [46] Chenxu Yang, Chuanyu Qin, Qingyi Si, Minghui Chen, Naibin Gu, Dingyu Yao, Zheng Lin, Weiping Wang, Jiaqi Wang, and Nan Duan. Self-Distilled RLVR, April 2026.
 - [47] Tianzhu Ye, Li Dong, Xun Wu, Shaohan Huang, and Furu Wei. On-Policy Context Distillation for Language Models, February 2026.
 - [48] Chen Zhang, Yang Yang, Jiahao Liu, Jingang Wang, Yunsen Xian, Benyou Wang, and Dawei Song. Lifting the Curse of Capacity Gap in Distilling Language Models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4535–4553, Toronto, Canada, 2023. Association for Computational Linguistics.
 - [49] Yifan Zhang and Team Math-AI. American invitational mathematics examination (AIME) 2025, 2025.
 - [50] Siyan Zhao, Zhihui Xie, Mengchen Liu, Jing Huang, Guan Pang, Feiyu Chen, and Aditya Grover. Self-Distilled Reasoner: On-Policy Self-Distillation for Large Language Models, January 2026.
 - [51] Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models, 2023.